

Monolith to Microservices: Step-by-Step Design & Implementation

Migrating from a **monolithic architecture** to **microservices** requires careful planning to ensure smooth transformation, maintainability, and scalability. Below is a **step-by-step approach**, including database strategies like **individual schema per service vs. shared schema** for related services.

◆ Step 1: Assess & Define Microservices Boundaries

- Identify modules in the monolith that can be **independently developed, deployed, and scaled**.
- Use **Domain-Driven Design (DDD)** principles:
 - **Bounded Contexts:** Define service boundaries based on business logic.
 - **Entities & Aggregates:** Identify key domain entities and relationships.

Example Breakdown (E-commerce System)

Monolith Module	Microservice Equivalent
Users	User Service
Orders	Order Service
Payments	Payment Service
Inventory	Inventory Service
Notifications	Notification Service

◆ Step 2: Choose Database Strategy

1 Database per Microservice (Recommended)

- Each microservice **owns its schema and data**.
- Ensures **loose coupling** and independent scaling.
- Uses **API calls or event-driven architecture** to sync data.

✓ **Best for:** Scalability, fault isolation.

✗ **Challenges:** Data consistency (use SAGA, CQRS).

◆ **Example:**

- `users_db`: Stores user profiles.
 - `orders_db`: Stores order details.
 - `payments_db`: Stores transactions.
-

2 Shared Database for Related Microservices

- Some **closely related services share a database schema**.
- **Foreign keys & joins are allowed** within shared services.
- Transactions & consistency are easier.

✓ **Best for:** Legacy systems with strong data relationships.

✗ **Challenges:** **Tight coupling**, limits independent scaling.

◆ **Example:**

- `order_service` & `payment_service` share `ecommerce_db`.
 - `product_service` & `inventory_service` share `inventory_db`.
-

◆ Step 3: Define Inter-Service Communication

Choose between:

- **Synchronous API Calls (REST/gRPC)** → Simple but increases dependencies.
- **Asynchronous Messaging (Kafka, RabbitMQ)** → Event-driven, more scalable.

✓ **Best Practice: Event-Driven Architecture** to decouple services.

◆ Example:

1. **Order Service Places an Order** → Publishes `OrderCreatedEvent`.
 2. **Inventory Service** listens & updates stock.
 3. **Payment Service** listens, processes payment & publishes `PaymentSuccessEvent`.
-

◆ Step 4: Implement Authentication & Authorization

- **Use JWT/OAuth2** for authentication.
- Implement **API Gateway** for centralized authentication.
- Enforce **RBAC (Role-Based Access Control)** per microservice.

◆ Example:

- `Auth Service` issues JWT tokens.
 - All services validate JWT before processing requests.
-

◆ Step 5: Implement Observability & Logging

- Use **Distributed Tracing** (Zipkin, Jaeger) to track requests.
 - Centralized **Logging & Monitoring** (ELK Stack, Prometheus + Grafana).
 - Implement **Circuit Breakers** (Resilience4j) to handle failures.
-

◆ Step 6: Gradual Migration Strategy

1. **Strangle Pattern**: Extract one module at a time into a microservice.
 2. **Feature Toggles**: Switch between monolith & microservices.
 3. **Data Sync Strategy**:
 - **Dual-write**: Sync data between monolith DB & new microservice DB.
 - **Event sourcing**: Maintain an event log for consistency.
-

Final Thoughts

- **Prefer individual schema per service** (decoupled, scalable).
- **Use shared schema sparingly** (only for tightly related microservices).
- **Implement event-driven design** for seamless microservice interaction.